

Questions de soutenance Vue.js

Fiche de révision - Options API, Composition API et projet Time Manager

Conseil : réponds d'abord avec tes propres mots, puis utilise la réponse courte pour vérifier les points importants.

1. Bases de Vue.js

1. Qu'est-ce que Vue.js ?

Réponse : Vue.js est un framework JavaScript progressif qui permet de créer des interfaces réactives et organisées en composants.

2. Qu'est-ce qu'un composant ?

Réponse : C'est une partie autonome de l'interface avec son affichage, ses données et sa logique. Exemple : `ClockManager.vue`.

3. Quelles sont les parties d'un fichier `.vue` ?

Réponse : `<template>` pour l'affichage, `<script>` pour la logique et `<style>` pour le CSS.

4. Comment utiliser un composant dans un autre ?

Réponse : On l'importe, on le déclare si nécessaire, puis on l'utilise comme une balise.

```
import ClockManager from './components/ClockManager.vue'
```

5. Qu'est-ce que la réactivité ?

Réponse : Quand une donnée réactive change, Vue met automatiquement à jour les éléments de l'interface qui en dépendent.

6. A quoi sert le template ?

Réponse : Il décrit ce que l'utilisateur voit et les interactions possibles.

7. A quoi sert `<v-if>` ?

Réponse : Il affiche un élément uniquement lorsque sa condition est vraie. L'élément est créé ou supprimé du DOM.

8. A quoi sert `<v-for>` ?

Réponse : Il parcourt une liste pour générer plusieurs éléments.

9. Pourquoi utiliser `:key` avec `<v-for>` ?

Réponse : Pour donner une identité stable à chaque élément et permettre à Vue de mettre à jour la liste correctement.

10. Quelle est la différence entre interpolation et liaison dynamique ?

Réponse : `{{ variable }}` affiche une valeur ; `:class`, `:disabled` ou `:to` lient dynamiquement une propriété HTML ou Vue.

2. Options API et Composition API

11. Qu'est-ce que l'Options API ?

Réponse : Elle organise un composant par catégories : `data`, `computed`, `methods`, `watch`, `props` et `hooks`.

12. Comment déclarer une donnée avec l'Options API ?

```
data() { return { count: 0 } }
```

13. Pourquoi utiliser `this` avec l'Options API ?

Réponse : `this` représente l'instance du composant et permet d'accéder à ses données et méthodes.

14. Qu'est-ce que la Composition API ?

Réponse : Elle regroupe la logique par fonctionnalité avec `ref`, `computed`, `watch` et les hooks de cycle de vie.

15. Comment créer un composant avec `<script setup>` ?

```
<script setup> import { ref } from 'vue' const count = ref(0) function increment() { count.value++ } </script>
```

16. Quelle est la différence entre `ref` et `reactive` ?

Réponse : `ref` convient à une valeur ou une référence ; `reactive` rend un objet réactif. Dans le script, un `ref` utilise `.value`.

17. Quelle API utilise `ClockManager.vue` ?

Réponse : L'Options API, car le composant utilise `data`, `computed`, `watch`, `methods` et `beforeUnmount`.

18. Quelle API est la meilleure ?

Réponse : Les deux sont supportées par Vue 3. L'Options API est claire pour débiter ; la Composition API facilite la réutilisation de logique dans les composants complexes.

19. Peut-on mélanger les deux API ?

Réponse : Oui, mais il vaut mieux rester cohérent dans un composant pour conserver un code lisible.

20. Qu'est-ce qu'un composable ?

Réponse : C'est une fonction basée sur la Composition API qui réutilise une logique, par exemple `useClock()` pour gérer un pointage.

3. Données et communication entre composants

21. Quelle est la différence entre ``data``, ``computed`` et ``methods`` ?

Réponse : ``data`` contient l'état, ``computed`` calcule des valeurs mémorisées, et ``methods`` contient des actions appelées explicitement.

22. Qu'est-ce qu'une prop ?

Réponse : C'est une donnée envoyée du parent vers l'enfant.

`props: { userId: { type: [Number, String], required: true } }`

23. Pourquoi `ClockManager` reçoit-il `userId` ?

Réponse : Pour savoir quel utilisateur rechercher et pour quel utilisateur créer un pointage.

24. Peut-on modifier directement une prop ?

Réponse : Non. Les props sont en lecture seule dans l'enfant. L'enfant doit émettre un événement pour demander au parent de modifier sa donnée.

25. À quoi sert ``$emit`` ?

Réponse : Il permet à l'enfant d'envoyer un événement au parent.

`this.$emit('changed')`

26. Que signifie cette utilisation ?

`<ClockManager :user-id="userId" @changed="onPeriodsChanged" />`

Réponse : Le parent transmet `userId` et écoute l'événement `changed`.

27. Pourquoi déclarer ``emits: ['changed']`` ?

Réponse : Pour déclarer les événements que le composant peut émettre et rendre son contrat plus clair.

28. Qu'est-ce que le two-way binding ?

Réponse : C'est une liaison dans les deux sens, souvent réalisée avec ``v-model``. La donnée est envoyée au composant et sa modification remonte au parent.

4. Cycle de vie, watch et computed

29. Qu'est-ce que le cycle de vie d'un composant ?

Réponse : Ce sont les étapes de création, montage, mise à jour et destruction du composant.

30. À quoi sert ``mounted`` ou ``onMounted`` ?

Réponse : À exécuter du code lorsque le composant est inséré dans la page.

31. À quoi sert ``beforeUnmount`` ?

Réponse : À nettoyer les ressources avant la destruction, par exemple arrêter le ``setInterval`` du timer.

32. À quoi sert ``watch`` ?

Réponse : À observer une donnée et exécuter une action lorsqu'elle change. Ici, le changement de `userId` déclenche ``refresh()``.

33. Que signifie ``immediate: true`` ?

Réponse : Le watcher est exécuté immédiatement au démarrage, sans attendre un premier changement.

34. Quand utiliser ``computed`` plutôt qu'une méthode ?

Réponse : Pour une valeur dérivée de données réactives. Vue la recalcule seulement lorsque ses dépendances changent.

35. Que fait le computed ``loading`` dans `ClockManager` ?

Réponse : Il vaut ``true`` si une lecture ou une sauvegarde est en cours : ``fetching || saving``.

5. API, erreurs et logique de ClockManager

36. Pourquoi utiliser ``async`` et ``await`` ?

Réponse : Les requêtes API sont asynchrones. ``await`` attend leur résultat avant de poursuivre la fonction.

37. Quel est le rôle de ``try/catch/finally`` ?

Réponse : ``try`` tente l'opération, ``catch`` traite l'erreur et ``finally`` s'exécute dans tous les cas.

38. Que fait ``refresh()`` ?

Réponse : Elle récupère les pointages, examine le dernier, met à jour ``clockIn`` et ``startDateTime``, puis démarre le timer si nécessaire.

39. Pourquoi utiliser ``ready`` ?

Réponse : Pour distinguer un état connu d'un état encore incertain. Le bouton est bloqué tant que l'état réel n'est pas récupéré.

40. Quelle est la différence entre ``fetching``, ``saving`` et ``ready`` ?

Réponse : ``fetching`` indique une lecture API, ``saving`` une sauvegarde, et ``ready`` indique que l'état obtenu est fiable.

41. Que fait ``status: !this.clockIn`` ?

Réponse : Le ``!`` inverse le booléen : hors service devient arrivée (``true``) et en service devient sortie (``false``).

42. Pourquoi utiliser ``requestVersion`` ?

Réponse : Pour ignorer une ancienne réponse API qui arriverait après une requête plus récente.

43. Pourquoi vérifier l'utilisateur dans ``isCurrentRequest`` ?

Réponse : Pour ne pas appliquer à l'utilisateur actuel une réponse qui concerne l'ancien utilisateur.

44. Pourquoi ne pas recharger toute la page après un pointage ?

Réponse : On rafraîchit seulement les données nécessaires, ce qui est plus rapide et conserve l'état de l'application.

45. Que faire après une erreur de sauvegarde ?

Réponse : Mettre l'état en doute avec ``ready = false``, afficher une erreur et de mander un rafraîchissement avant de réessayer, car l'enregistrement a peut-être réussi malgré l'erreur réseau.

Réponse globale à retenir :

Vue permet de construire une interface réactive avec des composants. Dans ce projet, ``App.vue`` coordonne l'application et ``ClockManager.vue`` gère le pointage, les appels API, les états de chargement et la communication avec le parent.